

EASYCRYPT tricks

Every listing below is a complete file from this folder. Each compiles on its own against the standard library, so any of them can be copied out, run, and changed.

```
for f in *.ec; do easycrypt compile "$f" || echo "FAILED: $f"; done
```

Contents

1	Rules of thumb	1
2	Reading the rules	2
3	conseq	2
4	call	4
5	exlim and seq	6
6	Probabilities	7
7	Up to bad	9
8	Dropping one-sided calls	12
9	Pitfalls	14
10	Seeing the goals from the command line	14

1 Rules of thumb

- **exlim** binds the value at the start of the remaining program. That is the value in the current precondition. If the program changes it before the point where you need it, cut the program first with **seq** or **sp**. **exlim** $e \Rightarrow v$ is the same as **exists*** e ; **elim*** $\Rightarrow v$.
- **Calls are processed from the end**. When dropping one-sided calls, handle the last one first.
- **An equiv cannot relate a side to its own past**. Fix the old value with **exlim** and add a hoare triple for that side with **conseq** $E \ H1 \ H2$.
- **byequiv lemma** needs an exact match. Write the spec you want in **byequiv** $(: P \Longrightarrow Q)$ and then **conseq lemma**. If the games are on the other sides, use **symmetry** first.
- **An up-to-bad invariant must include the equality of the flag**, or nothing says the left flag is still down while the right one is.
- **A fact stated as $\text{Pr}[\dots] = 1$ becomes a spec with bypr**. For a phoare spec, **bypr** $\Rightarrow \&m \leftarrow$; **exact** $(h \ \&m)$ is enough. For a hoare spec, go through **Pr**[mu_not] as in Section 6.

2 Reading the rules

Each section below states, as inference rules, the program-logic rule behind the tactics it shows. The rules are simplified to the shape the tactic uses, and they are read bottom-up, because the tactic turns the goal below the line into the goals above it.

- $\models c_1 \sim c_2 : \Phi \Rightarrow \Psi$ is a pRHL judgement. From any two memories related by Φ , the output distributions of c_1 and c_2 are related by Ψ , which means there is a coupling of them whose support lies in Ψ .
- Φ and Ψ relate two memories. For a formula P over one memory, $P\{1\}$ reads it in the left memory and $P\{2\}$ in the right one, and $=\{x\}$ abbreviates $x\{1\} = x\{2\}$.
- $\models c : P \Rightarrow Q$ is a Hoare judgement, saying that every terminating run of c from a memory in P ends in Q . It says nothing about termination.
- $\models c : P \Rightarrow Q \text{ } [=p]$ is a probabilistic Hoare judgement, saying that from every memory in P the probability that c ends in Q is exactly p . With $p = 1$ it gives both termination and the postcondition.
- $\text{mod}(c)$ is the set of variables c may write. When a rule quantifies over values for them, EASYCRYPT shows the quantified names as `result_L`, `0_R`, `S_R` and so on.

3 `conseq`

- `conseq` H applies a spec lemma and leaves two side goals, the precondition implication and the postcondition implication.
- `conseq` $(: _ \Rightarrow Q)$ changes the postcondition to Q and keeps the precondition. Use it to strengthen the postcondition to the equalities that `sim` proves.
- `conseq` $E \ H1 \ H2$ conjoins a hoare triple for each side onto a two-sided equiv. An equiv relates the two sides to each other, never a side to its own initial value, so facts like “the left counter went up by one” have to come from a one-sided hoare triple.
- $(_ : \text{true} \Rightarrow \text{true})$ in place of $H1$ or $H2$ means that nothing is added on that side. It becomes a trivial goal.
- `conseq` `ll` H turns losslessness and a hoare triple into a phoare spec with probability 1, which is what `call\{1\}` needs.
- `conseq` $H1 \ H2$ on a hoare goal conjoins the postconditions of two hoare triples.

$$\begin{array}{c}
 \frac{\Phi \Rightarrow \Phi' \quad \models c_1 \sim c_2 : \Phi' \Rightarrow \Psi' \quad \Psi' \Rightarrow \Psi}{\models c_1 \sim c_2 : \Phi \Rightarrow \Psi} \text{conseq} \\
 \\
 \frac{\models c_1 \sim c_2 : \Phi' \Rightarrow \Psi' \quad \models c_1 : P_1 \Rightarrow Q_1 \quad \models c_2 : P_2 \Rightarrow Q_2}{\models c_1 \sim c_2 : \Phi \Rightarrow \Psi} \text{conseq } E \ H1 \ H2 \\
 \\
 \frac{\models c : \text{true} \Rightarrow \text{true} \text{ } [=1] \quad \models c : P \Rightarrow Q}{\models c : P \Rightarrow Q \text{ } [=1]} \text{conseq } ll \ H
 \end{array}$$

The second rule is sound because a Hoare postcondition holds in every final memory of its side, and so in every pair of memories the coupling can reach. The last implication must hold in all such pairs, which is why EASYCRYPT states it with the variables in $\text{mod}(c_1)$ and $\text{mod}(c_2)$ universally quantified.

01_conseq.ec

```

1 (* conseq: the shapes worth knowing.
2  * Run: easycrypt compile 01_conseq.ec *)
3 require import AllCore.
4
5 module M = {
6   var x, y : int
7
8   proc incr() : unit = { x <- x + 1; }
9   proc both() : unit = { x <- x + 1; y <- y + 1; }
10  proc f(a : int) : int = { x <- x + 1; return a; }
11 }.
12
13 (* ----- *)

```

```

14 (* 1. Apply a spec lemma, weakening its post / strengthening its pre.
15 *   The side goals are  $pre \Rightarrow pre'$  and  $post' \Rightarrow post$ . *)
16 lemma incr_spec (n : int) : hoare [M.incr : M.x = n  $\Rightarrow$  M.x = n + 1].
17 proof. by proc; auto. qed.
18
19 lemma use_spec : hoare [M.incr : 0  $\leq$  M.x  $\Rightarrow$  0 < M.x].
20 proof.
21   exists* M.x; elim*  $\Rightarrow$  n.      (* name the initial value; see 03_elim_seq *)
22   by conseq (incr_spec n)  $\Rightarrow$  /#.
23 qed.
24
25 (* ----- *)
26 (* 2. Change the post to what the next tactic proves, keep the pre ( _ ).
27 *   Typical before sim, which wants equalities. *)
28 lemma to_sim : equiv [M.both ~ M.both :  $\{M.x, M.y\} \Rightarrow M.x\{1\} = M.x\{2\}$ ].
29 proof.
30   conseq ( : _  $\Rightarrow \{M.x, M.y\}$  )  $\Rightarrow$  //.
31   by sim.
32 qed.
33
34 (* Same, with the implication proved by hand instead of  $\Rightarrow$  // . *)
35 lemma to_sim' : equiv [M.both ~ M.both :  $\{M.x, M.y\} \Rightarrow M.x\{1\} + M.y\{1\} = M.x\{2\} + M.y\{2\}$ ].
36 proof.
37   conseq ( : _  $\Rightarrow \{M.x, M.y\}$  ).
38   + by move  $\Rightarrow$  &1 &2 /> /#.      (* post implication, if not automatic *)
39   by sim.
40 qed.
41
42 (* ----- *)
43 (* 3. conseq E H1 H2 : glue one-sided hoare facts onto a two-sided equiv.
44 *   An equiv relates  $\{1\}$  to  $\{2\}$ ; it cannot say how a side's state relates to
45 *   its OWN initial value. Fix the initial values, then add a hoare triple
46 *   per side. Side goals:  $pre \Rightarrow P \wedge P1\{1\} \wedge P2\{2\}$ , and
47 *    $Q \wedge Q1\{1\} \wedge Q2\{2\} \Rightarrow post$ . *)
48 equiv f_eq : M.f ~ M.f :  $\{arg\} \Rightarrow \{res\}$ .
49 proof. by proc; auto. qed.
50
51 lemma f_x (n : int) : hoare [M.f : M.x = n  $\Rightarrow$  M.x = n + 1].
52 proof. by proc; auto. qed.
53
54 lemma glue (n1 n2 : int) :
55   equiv [M.f ~ M.f :  $\{arg\} \wedge M.x\{1\} = n1 \wedge M.x\{2\} = n2$ 
56      $\Rightarrow \{res\} \wedge M.x\{1\} = n1 + 1 \wedge M.x\{2\} = n2 + 1$ ].
57 proof. by conseq f_eq (f_x n1) (f_x n2). qed.
58
59 (* ( _ : true  $\Rightarrow$  true) = "nothing extra on this side"; it becomes a trivial goal. *)
60 lemma glue_right (n2 : int) :
61   equiv [M.f ~ M.f :  $\{arg\} \wedge M.x\{2\} = n2 \Rightarrow \{res\} \wedge M.x\{2\} = n2 + 1$ ].
62 proof. by conseq f_eq ( _ : true  $\Rightarrow$  true) (f_x n2). qed.
63 (* Here  $\Rightarrow$  // closed the trivial hoare goal; otherwise: + by proc; auto. *)
64
65 (* Inside a proof, when the initial value is not a lemma argument yet:
66 *   exists* M.x{2}; elim*  $\Rightarrow$  n2.
67 *   conseq f_eq ( _ : true  $\Rightarrow$  true) (f_x n2). *)
68
69 (* ----- *)
70 (* 4. phoare = 1 from losslessness + a hoare triple (what call{i} wants). *)
71 lemma f_ll : islossless M.f.
72 proof. by proc; auto. qed.
73
74 lemma f_ph (n : int) : phoare [M.f : M.x = n  $\Rightarrow$  M.x = n + 1] = 1%r.
75 proof. by conseq f_ll (f_x n). qed.
76
77 (* ----- *)
78 (* 5. Combine two hoare triples (conjoin their posts). *)
79 lemma f_arg (a0 : int) : hoare [M.f : arg = a0  $\Rightarrow$  res = a0].
80 proof. by proc; auto. qed.
81
82 lemma f_both (n a0 : int) :
83   hoare [M.f : M.x = n  $\wedge$  arg = a0  $\Rightarrow$  M.x = n + 1  $\wedge$  res = a0].
84 proof. by conseq (f_x n) (f_arg a0)  $\Rightarrow$  /#. qed.

```

4 call

- `call (: I)` handles a call to an abstract adversary with an invariant I . It leaves one goal per oracle procedure, of the form $\models \{arg\} \wedge I \Rightarrow \{res\} \wedge I$. Directly on the procedure, the same thing is `proc I`.
- A procedure declared with `{}` in its module type may not call any oracle. `call (: true)` is then enough, since it only touches the adversary's own state and every other fact in the precondition carries over.
- For an abstract oracle procedure, or an alias like `proc get = O.get`, use `proc*; call (: true)`.
- `call (: I)` is only for abstract procedures. A concrete procedure needs a full spec, `call (: P  $\Rightarrow$  Q)`.
- `call{1} H` with H a phoare spec with probability 1 runs the call on the left side only. Its postcondition is all you learn about what the call did.
- `call H` with a hoare lemma H works as in hoare logic.

$$\frac{\forall o. \models O_1.o \sim O_2.o : \{arg\} \wedge I \Rightarrow \{res\} \wedge I}{\models A(O_1).f \sim A(O_2).f : \{arg, glob A\} \wedge I \Rightarrow \{res, glob A\} \wedge I} \text{ call } (: \quad I)$$

$$\frac{\begin{array}{c} \models f : P \Rightarrow Q \quad [=1] \\ \Phi \Rightarrow P[e/arg]\{1\} \wedge \forall v \vec{w}. Q[v/res, \vec{w}/mod(f)]\{1\} \Rightarrow \Psi[v/x, \vec{w}/mod(f)]\{1\} \end{array}}{\models x \leftarrow f(e) \sim skip : \Phi \Rightarrow \Psi} \text{ call}\{1\} \quad H$$

In the first rule o ranges over the oracle procedures A may call, I must not mention `glob A`, and A must not be able to write the variables of I . A procedure declared with `{}` may call no oracle, so the rule has no premises and $I = \text{true}$ suffices, while facts about variables outside `mod = glob A` survive the call unchanged.

In the second rule the new values of everything f may modify are universally quantified, and Q is all that is known about them. A spec that says nothing about some variable the rest of the proof needs leaves an unprovable goal, with a fresh name such as `S_R` where the old value used to be.

02_call.ec

```

1 (* call: the shapes worth knowing.
2  * Run: easyencrypt compile 02_call.ec *)
3 require import AllCore.
4
5 module type Orc = {
6   proc get(x : int) : int
7 }.
8
9 (* An adversary with one state-reading procedure that may NOT call the oracle
10  * ({ } = empty list of allowed oracle procedures) and one that may. *)
11 module type Adv (O : Orc) = {
12   proc save() : int { }
13   proc run() : bool
14 }.
15
16 module O1 : Orc = {
17   var n : int
18   proc get(x : int) : int = { n <- n + 1; return x; }
19 }.
20
21 module O2 : Orc = {
22   var m : int
23   proc get(x : int) : int = { m <- m + 1; return x; }
24 }.
25
26 (* An oracle that is a mere alias of another. *)
27 module W (O : Orc) : Orc = {
28   proc get = O.get
29 }.
30
31 module G (O : Orc) (A : Adv) = {
32   var s : int
33   proc main() : bool = {
34     var b;
35     s <-@ A(O).save();
36     b <-@ A(O).run();
37     return b;
38   }
39 }.

```

```

40
41 section.
42
43 declare module A <: Adv {-O1, -O2, -G}.
44 declare module O <: Orc {-A}.
45
46 (* ----- *)
47 (* 1. Abstract adversary, different oracles: call with an invariant I.
48 *   One goal per oracle procedure:  $\text{res} \wedge I \implies \text{res} \wedge I$ .
49 *   The call itself gives  $\text{res}, \text{glob } A \wedge I$  from  $\text{arg}, \text{glob } A \wedge I$ . *)
50 lemma run_rel :
51   equiv [G(O1, A).main ~ G(O2, A).main :
52      $\text{glob } A \wedge O1.n\{1\} = O2.m\{2\} \implies \text{res} \wedge O1.n\{1\} = O2.m\{2\}$ ].
53 proof.
54   proc.
55     call (: O1.n{1} = O2.m{2}).
56     + by proc; auto. (* the oracle goal *)
57     (* save has {} : it touches nothing but glob A, so (: true) is enough and
58      * every other fact in the precondition is carried over unchanged. *)
59     call (: true).
60     by auto.
61 qed.
62
63 (* The same statement proved directly on the procedure: proc I. *)
64 lemma run_rel' :
65   equiv [A(O1).run ~ A(O2).run :
66      $\text{glob } A \wedge O1.n\{1\} = O2.m\{2\} \implies \text{res}, \text{glob } A \wedge O1.n\{1\} = O2.m\{2\}$ ].
67 proof.
68   proc (O1.n{1} = O2.m{2})  $\Rightarrow$  //.
69   by proc; auto.
70 qed.
71
72 (* ----- *)
73 (* 2. An abstract oracle procedure, or an alias of one: proc* then call (: true).
74 *   For an abstract procedure call (: true) gives  $\text{res}, \text{glob } O$  from
75 *    $\text{arg}, \text{glob } O$ . *)
76 equiv w_get : W(O).get ~ W(O).get :  $\text{arg}, \text{glob } O \implies \text{res}, \text{glob } O$ .
77 proof. by proc; call (: true); auto. qed.
78
79 (* ----- *)
80 (* 3. One-sided call: call{1} / call{2} with a phoare = 1 spec.
81 *   The other side does nothing; the spec's post is all you learn. *)
82 module K = {
83   var c : int
84   proc tick() : unit = { c <- c + 1; }
85   proc two() : unit = { tick(); tick(); }
86   proc one() : unit = { tick(); }
87 }.
88
89 lemma tick_ph (n : int) : phoare [K.tick : K.c = n  $\implies$  K.c = n + 1] = 1%r.
90 proof. by proc; auto. qed.
91
92 lemma drop_one :
93   equiv [K.two ~ K.one :  $\text{K.c} \implies K.c\{1\} = K.c\{2\} + 1$ ].
94 proof.
95   proc.
96     (* last calls, two-sided. For a concrete procedure give a full spec
97      * (: P  $\implies$  Q); the invariant form (: I) is for abstract procedures. *)
98     call (: K.c{1} = K.c{2} + 1  $\implies$  K.c{1} = K.c{2} + 1).
99     + by proc; auto.
100     (* the remaining left-only call: fix K.c{1}, then call{1} *)
101     exlim K.c{1}  $\Rightarrow$  n.
102     call{1} (tick_ph n).
103     by auto  $\Rightarrow$  /#.
104 qed.
105
106 (* A lossless spec drops a call whose effect you don't care about:
107 *   call{1} some_ll. where some_ll : islossless P.f
108 *   You lose every fact about what P.f writes. *)
109
110 (* ----- *)
111 (* 4. call with a spec lemma instead of an invariant (hoare or equiv). *)
112 lemma tick_h (n : int) : hoare [K.tick : K.c = n  $\implies$  K.c = n + 1].
113 proof. by proc; auto. qed.
114

```

```

115 lemma two_h (n : int) : hoare [K.two : K.c = n ==> K.c = n + 2].
116 proof.
117   proc.
118     call (tick_h (n + 1)).
119     call (tick_h n).
120     by auto => /#.
121 qed.
122
123 end section.

```

5 exlim and seq

- `exlim e => v` binds `v` to the value of `e` in the precondition of the current goal. That precondition describes the state at the start of the program that is left, not at the point where you want to use `v`.
- The typical mistake is an `exlim` at the top of a proof whose program changes the value before the call that needs it. The call's precondition then asks for an equality that no longer holds.
- The fix is to cut the program first, so that the call becomes the start of what is left. `seq` cuts at any point, and `sp` consumes leading assignments.
- In a relational goal, say which side, as in `exlim x{1}, M.y{2} => a b`.
- An `exlim` at the top is fine when nothing before the call changes the value, because then the value at the start and the value at the call coincide.

$$\frac{\models c_1 \sim c_2 : \Phi \Rightarrow \Theta \quad \models c'_1 \sim c'_2 : \Theta \Rightarrow \Psi}{\models c_1; c'_1 \sim c_2; c'_2 : \Phi \Rightarrow \Psi} \text{seq} \qquad \frac{\forall v. \models c_1 \sim c_2 : \Phi \wedge e = v \Rightarrow \Psi}{\models c_1 \sim c_2 : \Phi \Rightarrow \Psi} \text{exlim } e \Rightarrow v$$

In the second rule v is fresh, and e is evaluated in the memories Φ talks about, which are the initial memories of all of c_1 and c_2 . To name a value in the middle of a program, apply `seq` first and then `exlim` in its second premise, where the middle of the program has become the start of c'_1 and c'_2 .

03_exlim_seq.ec

```

1 (* exlim / exists* and seq.
2  * Run: easycrypt compile 03_exlim_seq.ec
3  *
4  * THE RULE: exlim e => v binds v to the value of e in the PRECONDITION of
5  * the current goal, i.e. at the START of the program that is left. If the
6  * program changes e before the point where you need v, first cut the program
7  * so that point becomes the start (seq, sp), then exlim.
8  *
9  * exlim e1, e2 => a b.          same as exists* e1, e2; elim* => a b.
10 * exlim x{1}, M.y{2} => a b.   relational goals: say which side.
11 *)
12 require import AllCore.
13
14 module P = {
15   var x : int
16   proc inc() : unit = { x <- x + 1; }
17   proc main() : unit = { x <- 5; inc(); }
18   proc main2() : unit = { inc(); }
19 }.
20
21 lemma inc_h (n : int) : hoare [P.inc : P.x = n ==> P.x = n + 1].
22 proof. by proc; auto. qed.
23
24 lemma inc_ph (n : int) : phoare [P.inc : P.x = n ==> P.x = n + 1] = 1%r.
25 proof. by proc; auto. qed.
26
27 (* ----- *)
28 (* WRONG: exlim at the top binds the value BEFORE x <- 5, so the call's
29  * precondition P.x = n cannot be met:
30  *
31  *   proc. exlim P.x => n. call (inc_h n).   (* stuck: need 5 = n *)
32  *
33  * RIGHT: cut first, so the call is at the start of what is left.      *)
34 lemma main_h : hoare [P.main : true ==> P.x = 6].
35 proof.

```

```

36   proc.
37   seq 1 : (P.x = 5); first by auto.      (* hoare seq: seq <#stmts> : (P) *)
38   exlim P.x ⇒ n.                        (* now n is the value at the call *)
39   call (inc_h n).
40   by auto ⇒ /#.
41 qed.
42
43 (* For straight-line assignments at the start, sp does the cut for you. *)
44 lemma main_h' : hoare [P.main : true ⇒ P.x = 6].
45 proof.
46   proc; sp.                             (* pre is now P.x = 5 *)
47   exlim P.x ⇒ n.
48   call (inc_h n).
49   by auto ⇒ /#.
50 qed.
51
52 (* ----- *)
53 (* Relational version: a one-sided call needs the ghost at the call point.
54  * equiv seq: seq <#left> <#right> : (P). #pre = the current precondition. *)
55 lemma main_eq : equiv [P.main ~ P.main2 : true ⇒ P.x{1} = 6].
56 proof.
57   proc.
58   seq 1 0 : (P.x{1} = 5); first by auto.
59   exlim P.x{1} ⇒ n.
60   call{1} (inc_ph n).
61   (* right side still has its call; drop it one-sided too (no ghost needed) *)
62   call{2} (: true ⇒ true); first by proc; auto.
63   by auto ⇒ /#.
64 qed.
65
66 (* ----- *)
67 (* Calls are processed from the END of the program. exlim at the top is fine
68  * when nothing before the call changes the value: then "value at the start"
69  * and "value at the call" coincide. *)
70 lemma twice (n : int) :
71   equiv [P.main2 ~ P.main2 : P.x{1} = n ∧ P.x{2} = n ⇒ ={P.x}].
72 proof.
73   proc.
74   exlim P.x{1} ⇒ v.                      (* fine: inc is the first statement *)
75   call{1} (inc_ph v).
76   exlim P.x{2} ⇒ w.
77   call{2} (inc_ph w).
78   by auto ⇒ /#.
79 qed.
80
81 (* ----- *)
82 (* Other cutting tools
83  * seq 3 3 : (#pre ∧ ={x}).             keep the old facts and add one
84  * seq 1 1 : (Q); first by ... .         discharge the prefix inline
85  * sp / sp 1 1                           consume leading assignments into the pre
86  * wp                                       consume trailing assignments into the post
87  * swap{1} 3 -2                           move left statement 3 up by 2 (needs
88  *                                         syntactic independence) *)

```

6 Probabilities

- `rewrite Pr[lemma]` applies a distribution lemma to a probability term. The useful ones are `mu_le1`, `mu_sub`, `mu_not`, `mu_or` and `mu_split`. `E`.
- $\Pr[E] = 0$ follows from a hoare triple on $\neg E$, by `byphoare`, then `hoare`.
- A phoare spec with probability 1 follows from a fact of the form $\Pr[\dots] = 1$ that holds at every memory, by `bypr`.
- A hoare spec follows from the same fact. `bypr` turns the goal into $\Pr[\neg Q] = 0$, and $\Pr[\neg Q] = \Pr[true] - \Pr[Q]$ with $1 = \Pr[Q] \leq \Pr[true] \leq 1$. No losslessness is needed.
- Inside an event written in a proof, `X{m}` for the memory bound by `bypr` is read as the current `X`. Rather than retyping the event, rewrite with the hypothesis that already contains it, as in `rewrite -h1 Pr[mu_sub]`.
- `byequiv lemma` needs the lemma's pre- and postcondition to match exactly. State the spec in `byequiv`, discharge it with `conseq`, and use `symmetry` when the games sit on the other sides.
- `smt` treats probability terms as opaque reals, so arithmetic over several of them works as long as the terms are syntactically identical.

$$\begin{array}{c}
\frac{\models c_1 \sim c_2 : \Phi \Rightarrow \Psi \quad \Phi(m_1, m_2) \quad \Psi \Rightarrow (E_1\{1\} \Rightarrow E_2\{2\})}{\Pr[c_1, m_1 : E_1] \leq \Pr[c_2, m_2 : E_2]} \text{byequiv} \\
\\
\frac{\frac{\models c_2 \sim c_1 : \Phi^{-1} \Rightarrow \Psi^{-1}}{\models c_1 \sim c_2 : \Phi \Rightarrow \Psi} \text{symmetry} \quad \frac{\forall m. P(m) \Rightarrow \Pr[c, m : Q] = p}{\models c : P \Rightarrow Q \text{ } [=p]} \text{bypr (phoare)}}{\frac{\forall m. P(m) \Rightarrow \Pr[c, m : \neg Q] = 0}{\models c : P \Rightarrow Q} \text{bypr (hoare)} \quad \frac{\models c : P \Rightarrow \neg E \quad P(m)}{\Pr[c, m : E] = 0} \text{byphoare; hoare}}
\end{array}$$

With $\Leftrightarrow$ in place of $\Rightarrow$ in the last premise, **byequiv** gives equality. Φ^{-1} is Φ with $\{1\}$ and $\{2\}$ exchanged.

04_pr.ec

```

1 (* Pr[...] rewriting, and moving between Pr, hoare and phoare.
2  * Run: easycrypt compile 04_pr.ec *)
3 require import AllCore Distr DBool.
4
5 module T = {
6   var x : int
7   proc flip() : bool = { var b; b <$ {0,1}; return b; }
8   proc keep() : unit = { }
9 }.
10
11 (* ----- *)
12 (* 1. The rewrite rules: Pr[lemma] applies a distribution lemma to a Pr term. *)
13 lemma le1 &m : Pr[T.flip() @ &m : res] ≤ 1%r.
14 proof. by rewrite Pr[mu_le1]. qed.
15
16 lemma sub &m : Pr[T.flip() @ &m : res ∧ T.x = 0] ≤ Pr[T.flip() @ &m : res].
17 proof. by rewrite Pr[mu_sub]. qed. (* event inclusion *)
18
19 lemma not &m :
20   Pr[T.flip() @ &m : !res] = Pr[T.flip() @ &m : true] - Pr[T.flip() @ &m : res].
21 proof. by rewrite Pr[mu_not]. qed.
22
23 lemma or &m :
24   Pr[T.flip() @ &m : res ∨ T.x = 0] =
25   Pr[T.flip() @ &m : res] + Pr[T.flip() @ &m : T.x = 0]
26   - Pr[T.flip() @ &m : res ∧ T.x = 0].
27 proof. by rewrite Pr[mu_or]. qed.
28
29 lemma split_ex &m :
30   Pr[T.flip() @ &m : res] =
31   Pr[T.flip() @ &m : res ∧ T.x = 0] + Pr[T.flip() @ &m : res ∧ !(T.x = 0)].
32 proof. by rewrite Pr[mu_split (T.x = 0)]. qed.
33
34 (* NOTE: `split` is a keyword: `lemma split` and `have split : ...` are both
35  * parse errors. Same for other tactic names. Use hsplit, split_ex, ... *)
36
37 (* ----- *)
38 (* 2. Pr[E] = 0 from a hoare triple on "not E". *)
39 lemma never &m : Pr[T.keep() @ &m : T.x ≠ T.x{m}] = 0%r.
40 proof. byphoare (_, T.x = T.x{m} => _) => //; hoare; proc; auto. qed.
41
42 (* ----- *)
43 (* 3. phoare = 1 from a Pr = 1 fact that holds at every memory: bypr. *)
44 lemma keep_pr &m : Pr[T.keep() @ &m : T.x = T.x{m}] = 1%r.
45 proof. byphoare (_, T.x = T.x{m} => T.x = T.x{m}) => //; proc; auto. qed.
46
47 lemma keep_ph (n : int) : phoare [T.keep : T.x = n => T.x = n] = 1%r.
48 proof. by bypr => &m <-; exact (keep_pr &m). qed.
49
50 (* ----- *)
51 (* 4. hoare from a Pr = 1 fact: bypr turns the goal into Pr[not Q] = 0.
52  * Pr[not Q] = Pr[true] - Pr[Q] = Pr[true] - 1, and 1 = Pr[Q] ≤ Pr[true] ≤ 1.
53  * No losslessness needed.
54  *
55  * PITFALL: inside a Pr[...] event written in a proof, T.x{m} for the
56  * memory bound by bypr is read as the CURRENT value (the event becomes
57  * T.x = T.x). Don't write the event by hand; get it from the hypothesis,
58  * as in rewrite -h1 Pr[mu_sub]. *)

```

```

59 lemma keep_h (n : int) : hoare [T.keep : T.x = n ==> T.x = n].
60 proof.
61   bypr => &m <-.
62   have h1 := keep_pr &m.
63   have h2 : Pr[T.keep() @ &m : true] ≤ 1%r by rewrite Pr[mu_le1].
64   have h3 : 1%r ≤ Pr[T.keep() @ &m : true] by rewrite -h1 Pr[mu_sub].
65   by rewrite Pr[mu_not] h1; smt().
66 qed.
67
68 (* ----- *)
69 (* 5. byequiv: a lemma must match the goal's pre/post EXACTLY. State the
70 *   spec you want in byequiv, then conseq the lemma into it. If the games
71 *   are on the other sides, symmetry flips the goal ({1} and {2} swap). *)
72 module U = {
73   proc f() : bool = { var b; b <$ {0,1}; return b; }
74   proc g() : bool = { var b; b <$ {0,1}; return !b; }
75 }.
76
77 equiv fg : U.f ~ U.g : true ==> res{1} = !res{2}.
78 proof. by proc; auto. qed. (* couples b{1} = b{2} *)
79
80 lemma fg_pr &m : Pr[U.f() @ &m : res] ≤ Pr[U.g() @ &m : !res].
81 proof.
82   byequiv (: true ==> res{1} = !res{2}) => //. (* => // closed pre/post *)
83   by conseq fg.
84 qed.
85
86 lemma gf_pr &m : Pr[U.g() @ &m : !res] ≤ Pr[U.f() @ &m : res].
87 proof.
88   byequiv (: true ==> res{2} = !res{1}) => //.
89   by symmetry; conseq fg => /#.
90 qed.
91
92 (* ----- *)
93 (* 6. Pr arithmetic with smt: hypotheses about Pr terms are fine, smt treats
94 *   the Pr terms as opaque reals. Keep the terms syntactically identical. *)
95 lemma eq_from_two &m : Pr[U.f() @ &m : res] = Pr[U.g() @ &m : !res].
96 proof. by have := fg_pr &m; have := gf_pr &m; smt(). qed.

```

7 Up to bad

The call rule takes three components, `call` (: B, I, J).

B	the bad event, read on the right memory and written without {2}
I	the relational invariant while $\neg B$, which must include the equality of the flag
J	what still holds once B is set, preserved by each side alone (optional)

It generates a goal saying that the adversary terminates when its oracles do, and then three goals for every oracle procedure `o`.

1. `equiv` [`o ~ o : !B{2} ∧ ={arg} ∧ I ==> if B{2} then J else ={res} ∧ I`].
2. `forall` &2, `B{2} => phoare` [`o{1} : J ==> J`] = 1%r, for the left side after bad.
3. `forall` &1, `phaore` [`o{2} : B ∧ J ==> B ∧ J`] = 1%r, which says the right side keeps bad.

The code before the call is left with the postcondition `if B{2} then J else (= {res, glob A} ∧ I)`.

- Choose J so that it implies bad stays set. If some oracle could unset bad, for instance by overwriting the record that bad is computed from, put into J whatever switches that oracle off.
- If you only need `!B{2} => ={res}` and not the equality of the flags afterwards, drop J. The goals after bad then only ask for termination.
- From the equivalence, one `byequiv` gives $\Pr_1[E] \leq \Pr_2[E \vee \text{bad}] \leq \Pr_2[E] + \Pr_2[\text{bad}]$.
- If $\Pr_2[\text{bad}] = 0$, which a hoare triple can show, a second `byequiv` in the other direction gives $\Pr_2[E \wedge \neg \text{bad}] \leq \Pr_1[E]$. Splitting $\Pr_2[E]$ on bad then yields equality.

Writing $\mathcal{I}(X)$ for `if B{2} then J else (= {X} ∧ I)`, the rule reads as follows.

$$\begin{array}{c}
\text{A is lossless when its oracles are} \\
\frac{\forall o. \models 0_1.o \sim 0_2.o : \neg B\{2\} \wedge \{arg\} \wedge I \Rightarrow \mathcal{I}(res) \quad \forall o. \forall m_2. B(m_2) \Rightarrow (\models 0_1.o : J \Rightarrow J [=1]) \quad \forall o. \forall m_1. \models 0_2.o : B \wedge J \Rightarrow B \wedge J [=1]}{\models A(0_1).f \sim A(0_2).f : \mathcal{I}(arg, glob A) \Rightarrow \mathcal{I}(res, glob A)} \text{ call } (: B, I, J) \\
\\
\frac{\models c_1 \sim c_2 : \Phi \Rightarrow \{B\} \wedge (\neg B\{2\} \Rightarrow \{res\}) \quad \Phi(m_1, m_2)}{\Pr[c_1, m_1 : E] \leq \Pr[c_2, m_2 : E] + \Pr[c_2, m_2 : B]} \text{ up to bad}
\end{array}$$

In the third and fourth premises J is a relation, and the memory of the side that does not run is the fixed m_2 or m_1 . The second rule is **byequiv** followed by $\Pr[E \vee B] \leq \Pr[E] + \Pr[B]$, where E is an event on the result.

05_upto_bad.ec

```

1 (* Up-to-bad: two games that agree until a flag is raised.
2 * Run: easyencrypt compile 05_upto_bad.ec
3 *
4 * The call rule has three parts:
5 *
6 *   call (: B, I, J).
7 *   B : the bad event, read on the RIGHT memory (written without {2})
8 *   I : relational invariant while not B
9 *   J : what still holds once B happened (relational; optional)
10 *
11 * It generates, for the adversary and then for EACH oracle procedure o:
12 *   - the adversary terminates when its oracles do
13 *   - equiv [o ~ o : !B{2} ∧ {arg} ∧ I ⇒ if B{2} then J else {res} ∧ I]
14 *   - forall @2, B{2} ⇒ phoare [o{1} : J ⇒ J] = 1      (left, after bad)
15 *   - forall @1,      phoare [o{2} : B ∧ J ⇒ B ∧ J] = 1 (right keeps bad)
16 * and leaves the code before the call with post
17 *   if B{2} then J else ({res, glob A} ∧ I).
18 *
19 * Choosing I: include {flag} itself, or nothing says the LEFT flag is
20 * still down while the right one is (you then can't conclude {Flag.bad}).
21 *
22 * Choosing J: it must be preserved by each side ALONE, and it must imply
23 * that B stays set. If an oracle could un-set B (e.g. overwrite the record
24 * B is computed from) put in J whatever switches that oracle off.
25 *)
26 require import AllCore Distr.
27
28 module type Orc = {
29   proc f(x : int) : int
30 }.
31
32 module type Adv (O : Orc) = {
33   proc run() : bool
34 }.
35
36 module Flag = {
37   var bad : bool
38   var s   : int
39 }.
40
41 (* Two oracles that differ only on the point s; both raise the flag there.
42 * Nonnegative queries only, so with s < 0 the flag can never be raised. *)
43 module OL : Orc = {
44   proc f(x : int) : int = {
45     if (0 ≤ x ∧ x = Flag.s) { Flag.bad <- true; }
46     return x;
47   }
48 }.
49
50 module OR : Orc = {
51   proc f(x : int) : int = {
52     var r;
53     r <- x;
54     if (0 ≤ x ∧ x = Flag.s) { Flag.bad <- true; r <- 0; }
55     return r;
56   }
57 }.

```

```

58
59 module Game (O : Orc) (A : Adv) = {
60   proc main(s0 : int) : bool = {
61     var b;
62     Flag.bad <- false;
63     Flag.s <- s0;
64     b <@ A(O).run();
65     return b;
66   }
67 }.
68
69 section.
70
71 declare module A <: Adv {-Flag}.
72
73 declare axiom A_ll (O <: Orc {-A}) : islossless O.f ⇒ islossless A(O).run.
74
75 (* ----- *)
76 (* 1. The up-to-bad equivalence. *)
77 equiv upto : Game(OL, A).main ~ Game(OR, A).main :
78   ={glob A, arg} ⇒ ={Flag.bad} ∧ (!Flag.bad{2} ⇒ ={res}).
79 proof.
80   proc.
81     call (: Flag.bad, ={Flag.s, Flag.bad}, Flag.bad{1}).
82     + exact A_ll. (* adversary terminates *)
83     + by proc; auto ⇒ /#. (* oracle, before bad *)
84     + by move⇒ &2 _; proc; auto. (* left oracle, after bad *)
85     + by move⇒ &1; proc; auto. (* right oracle keeps bad *)
86     by auto ⇒ /#.
87 qed.
88
89 (* If you only need !bad{2} ⇒ ={res} (not ={Flag.bad}), drop J:
90 * call (: Flag.bad, ={Flag.s, Flag.bad}).
91 * and the "after bad" goals only ask for termination. *)
92
93 (* ----- *)
94 (* 2. The usual bound: Pr1[E] ≤ Pr2[E] + Pr2[bad]. *)
95 lemma upto_bound &m (s0 : int) :
96   Pr[Game(OL, A).main(s0) @ &m : res] ≤
97   Pr[Game(OR, A).main(s0) @ &m : res] + Pr[Game(OR, A).main(s0) @ &m : Flag.bad].
98 proof.
99   have le : Pr[Game(OL, A).main(s0) @ &m : res] ≤
100     Pr[Game(OR, A).main(s0) @ &m : res ∨ Flag.bad].
101   + byequiv (: ={glob A, arg} ⇒ ={Flag.bad} ∧ (!Flag.bad{2} ⇒ ={res})) ⇒ //.
102   + by conseq upto.
103   by move⇒ &1 &2 /#.
104   have hor : Pr[Game(OR, A).main(s0) @ &m : res ∨ Flag.bad] ≤
105     Pr[Game(OR, A).main(s0) @ &m : res] +
106     Pr[Game(OR, A).main(s0) @ &m : Flag.bad].
107   + by rewrite Pr[mu_or]; smt(mu_bounded). (* drop the - Pr[res ∧ bad] *)
108   smt().
109 qed.
110
111 (* ----- *)
112 (* 3. When bad provably never happens: Pr[bad] = 0 from a hoare triple on the
113 * right game, then equality from two one-directional byequivs. *)
114 lemma no_bad &m (s0 : int) :
115   s0 < 0 ⇒ Pr[Game(OR, A).main(s0) @ &m : Flag.bad] = 0%r.
116 proof.
117   move⇒ hs; byphoare (: arg = s0 ⇒ _) ⇒ //; hoare.
118   proc.
119   call (: !Flag.bad ∧ Flag.s < 0); first by proc; auto ⇒ /#.
120   by auto ⇒ /#.
121 qed.
122
123 lemma upto_eq &m (s0 : int) :
124   s0 < 0 ⇒
125   Pr[Game(OL, A).main(s0) @ &m : res] = Pr[Game(OR, A).main(s0) @ &m : res].
126 proof.
127   move⇒ hs; have nb := no_bad &m s0 hs.
128   (* Pr1[res] ≤ Pr2[res ∨ bad] *)
129   have le1 : Pr[Game(OL, A).main(s0) @ &m : res] ≤
130     Pr[Game(OR, A).main(s0) @ &m : res ∨ Flag.bad].
131   + byequiv (: ={glob A, arg} ⇒ ={Flag.bad} ∧ (!Flag.bad{2} ⇒ ={res})) ⇒ //.
132   + by conseq upto.

```

```

133   by move⇒ &1 &2 /#.
134   (* Pr2[res ∧ !bad] ≤ Pr1[res]: the games are now on the other sides *)
135   have le2 : Pr[Game(OR, A).main(s0) @ &m : res ∧ !Flag.bad] ≤
136     Pr[Game(OL, A).main(s0) @ &m : res].
137   + byequiv (:= {glob A, arg} ⇒ {Flag.bad} ∧ (!Flag.bad{1} ⇒ {res})) ⇒ //.
138   + by symmetry; conseq upto ⇒ /#.
139   by move⇒ &1 &2 /#.
140   (* arithmetic *)
141   have hor : Pr[Game(OR, A).main(s0) @ &m : res ∨ Flag.bad] ≤
142     Pr[Game(OR, A).main(s0) @ &m : res] +
143     Pr[Game(OR, A).main(s0) @ &m : Flag.bad].
144   + by rewrite Pr[mu_or]; smt(mu_bounded).
145   have hsplit : Pr[Game(OR, A).main(s0) @ &m : res] =
146     Pr[Game(OR, A).main(s0) @ &m : res ∧ Flag.bad] +
147     Pr[Game(OR, A).main(s0) @ &m : res ∧ !Flag.bad].
148   + by rewrite Pr[mu_split Flag.bad].
149   have hsub : Pr[Game(OR, A).main(s0) @ &m : res ∧ Flag.bad] ≤
150     Pr[Game(OR, A).main(s0) @ &m : Flag.bad].
151   + by rewrite Pr[mu_sub].
152   smt(mu_bounded).
153 qed.
154
155 end section.

```

8 Dropping one-sided calls

A rewinding reduction runs the adversary once more than the game it is compared with, and that extra run is dropped on one side only.

1. For every call you drop, get a phoare spec with probability 1 that says what it does to the state you still care about. It comes from losslessness and a hoare fact through `conseq` 11 H, or from an axiom of the form $\text{Pr}[\dots] = 1$ through `bypr`. If the axiom is existential, as a rewinding axiom usually is, these specs must be `haves` inside the proof, after `elim`.
2. Fix, with `exlim`, the values those specs need, at a point where they are right.
3. Run `call{1}` with each spec, from the last dropped call backwards.
4. Close with `auto` and `smt`.

The state-reading procedures are declared with `{}`, so they cannot call the oracle. They stand for the reduction taking a snapshot of the adversary, not for something the adversary does.

Each dropped call is one application of `call{1}` H from Section 4. For a discarded run between a save and a restore, the derivation is the following chain, read from the bottom.

$$\begin{array}{c}
\frac{\Phi \wedge \text{glob } A\{1\} = g \Rightarrow \Psi}{\vdash \text{load}(s) \sim \text{skip} : s\{1\} = f(g) \wedge \Phi \Rightarrow \Psi} \text{ call}\{1\} \text{ (phl } g) \\
\frac{\vdash \text{run}(); \text{load}(s) \sim \text{skip} : s\{1\} = f(g) \wedge \Phi \Rightarrow \Psi}{\vdash s \leftarrow \text{save}(); \text{run}(); \text{load}(s) \sim \text{skip} : \text{glob } A\{1\} = g \wedge \Phi \Rightarrow \Psi} \text{ call}\{1\} \text{ (phr } k) \\
\vdash s \leftarrow \text{save}(); \text{run}(); \text{load}(s) \sim \text{skip} : \text{glob } A\{1\} = g \wedge \Phi \Rightarrow \Psi \quad \text{ call}\{1\} \text{ (phs } g)
\end{array}$$

Here Φ mentions only state that `run` leaves alone, which is what the spec `phr` provides, and `load(f(g))` restores `glob A` to `g` whatever `run` did to it.

O6_one_sided.ec

```

1  (* Dropping one-sided calls to an abstract adversary (e.g. a discarded run in
2  * a rewinding reduction).
3  * Run: easycrypt compile O6_one_sided.ec
4  *
5  * Recipe:
6  *   1. for every call you drop, a phoare [...] = 1%r spec saying what it does
7  *      to the state you still care about;
8  *   2. elim the values those specs need, at a point where they are right
9  *      (see O3_elim_seq.ec);
10 *   3. call{1} each spec, from the LAST dropped call backwards;
11 *   4. close with auto / smt.
12 *
13 * Where the phoare specs come from:

```

```

14 * - losslessness + a hoare fact:   $\text{conseq } ll \ H \quad (O1\_conseq.ec, 4.)$ 
15 * - an axiom stated with  $\text{Pr}[\dots] = 1\%r$ :   $\text{bypr} \Rightarrow \&m \dots; \text{exact } \dots$ 
16 *)
17 require import AllCore.
18
19 module type Orc = {
20   proc get(x : int) : int
21 }.
22
23 (* save and load may not call the oracle ({}): they are the reduction's
24   * snapshot of the adversary, not adversary actions. *)
25 module type Adv (O : Orc) = {
26   proc save() : int {}
27   proc load(s : int) : unit {}
28   proc run() : bool
29 }.
30
31 (* An oracle that only reads its state, so a run leaves it unchanged. *)
32 module Cnt : Orc = {
33   var k : int
34   proc get(x : int) : int = { return x + k; }
35 }.
36
37 module G1 (A : Adv) = {
38   var st : int
39   proc main() : bool = {
40     var b1, b2;
41     st <@ A(Cnt).save();
42     b1 <@ A(Cnt).run();           (* discarded run *)
43     A(Cnt).load(st);
44     b2 <@ A(Cnt).run();
45     return b2;
46   }
47 }.
48
49 module G2 (A : Adv) = {
50   proc main() : bool = {
51     var b;
52     b <@ A(Cnt).run();
53     return b;
54   }
55 }.
56
57 section.
58
59 declare module A <: Adv {-Cnt, -G1}.
60
61 declare axiom A_ll (O <: Orc {-A}) : islossless O.get  $\Rightarrow$  islossless A(O).run.
62
63 (* Rewindability, stated with probabilities. *)
64 declare axiom A_rew :
65   exists (f : glob A  $\rightarrow$  int),
66     (forall (O <: Orc) &m,
67       Pr[A(O).save() @ &m : (glob A) = (glob A){m}  $\wedge$  res = f (glob A){m}] = 1%r)  $\wedge$ 
68       (forall (O <: Orc) &m (x : glob A) (s : int),
69         s = f x  $\Rightarrow$  Pr[A(O).load(s) @ &m : (glob A) = x] = 1%r).
70
71 lemma drop_run : equiv [G1(A).main ~ G2(A).main : ={glob A, Cnt.k}  $\Longrightarrow$  ={res}].
72 proof.
73   (* 1. the specs. f only exists after elim, so these are haves. *)
74   elim A_rew  $\Rightarrow$  f [hs hl].
75   have phs : forall g, phoare [A(Cnt).save : (glob A) = g  $\Longrightarrow$  (glob A) = g  $\wedge$  res = f g] = 1%r.
76   + by move $\Rightarrow$  g; bypr  $\Rightarrow$  &m <-; exact (hs Cnt &m).
77   have phl : forall g, phoare [A(Cnt).load : arg = f g  $\Longrightarrow$  (glob A) = g] = 1%r.
78   + by move $\Rightarrow$  g; bypr  $\Rightarrow$  &m hs'; exact (hl Cnt &m g arg{m} hs').
79   have phr : forall k0, phoare [A(Cnt).run : Cnt.k = k0  $\Longrightarrow$  Cnt.k = k0] = 1%r.
80   + move $\Rightarrow$  k0; conseq (A_ll Cnt _) ( _ : Cnt.k = k0  $\Longrightarrow$  Cnt.k = k0)  $\Rightarrow$  //.
81   (* goals come out in this order: the hoare fact, then the side condition
82     * of A_ll. When unsure, look: easycrypt llm, then GOALS ALL. *)
83   + by proc (Cnt.k = k0)  $\Rightarrow$  //; proc; auto.      (* run leaves Cnt.k *)
84   by proc; auto.      (* Cnt.get is lossless *)
85   proc.
86   (* the kept run, two-sided *)
87   call (: ={Cnt.k}); first by proc; auto.
88   (* 2. nothing before the dropped calls changes glob A or Cnt.k, so elim

```

```

89  *    at the top gives the right values *)
90  exlim (glob A){1}, Cnt.k{1} => g k0.
91  (* 3. backwards: load, then the run, then save *)
92  call{1} (phl g).
93  call{1} (phr k0).
94  call{1} (phs g).
95  (* 4. *)
96  by auto => /#.
97  qed.
98
99  end section.

```

9 Pitfalls

- `forall` extends as far to the right as it can. In an invariant write $\wedge (\text{forall } y, P \ y) \wedge Q$ with the parentheses, or Q ends up under the binder.
- `split` and other tactic names cannot be used as lemma or hypothesis names.
- Program expressions cannot mention `glob` M . EASYCRYPT reports that expressions cannot contain a `glob` statement.
- Inside a section, a `declare axiom` cannot use a `local` predicate, and a non-local operator cannot mention the `glob` of a declared module. Spell the predicate out in the axiom.
- `=> //` may close more goals than expected, and the next bullet then fails because all goals are closed.
- `ecall{1}` (`lemma e{1}`) crashed with `InvalidGoalShape` in the version used here. `seq`, `exlim` and `call{1}` do the same job.
- The order of the side goals from `conseq` is not always the order of its arguments. Look at them before writing bullets.

10 Seeing the goals from the command line

`easycrypt llm` loads a file up to a line and prints the goals there.

```

easycrypt llm -I <dirs> -eval 'LOAD "file.ec" 42
QUIET ON
proc.
GOALS ALL'

```

- `LOAD` checks everything up to and including the given line, then waits for more input. If that line is `proof`, `admit`, `qed`., split it first, or the load runs past the proof.
- Every further line is one tactic, one sentence per line.
- `GOALS` prints the focused goal, `GOALS ALL` prints all of them, and `TREE ALL` prints the goal tree. An error prints `ERROR` together with the offending `source:` line.